

***DANIYA ALI***

***CT – 25159***

***LAB NO 3 (TYPES OF LINKED LIST)***

***INSTRUCTOR: MR. MUDASSIR AHMED***

***COURSE CODE : CT-159***

## QUESTION NO 1

SOURCE CODE:

```
1  #include <iostream>
2  using namespace std;
3
4  struct Node {
5      int data;
6      Node* next;
7
8      Node(int val) {
9          data = val;
10         next = nullptr;
11     }
12 };
13
14 class CircularQueue {
15 private:
16     Node* rear;
17
18 public:
19     CircularQueue() {
20         rear = nullptr;
21     }
22
23     bool isEmpty() {
24         return rear == nullptr;
25     }
26
27     //Insert an element at the rear of the queue
28     void enqueue(int value) {
29         Node* newNode = new Node(value);
30         if (isEmpty()) {
31             rear = newNode;
32             rear->next = rear; // points to itself to maintain the circular link
33         } else {
34             newNode->next = rear->next; // new node points to front
35             rear->next = newNode;      // rear next is now newNode
36             rear = newNode;           // Update rear to the new node
37         }
38         cout << value << " inserted into the queue." << endl;
39     }
40
41     //Remove an element from the front of the queue
42     void dequeue() {
43         if (isEmpty()) {
44             cout << "Queue Underflow! The queue is empty." << endl;
45             return;
46         }
47     }
```

```

45         return;
46     }
47
48     Node* front = rear->next;
49     cout << front->data << " removed from the queue." << endl;
50
51     if (rear == front) { // Only one element in the queue
52         delete rear;
53         rear = nullptr;
54     } else {
55         rear->next = front->next;
56         delete front;
57     }
58 }
59
60 // Get the front element of the queue
61 int peek() {
62     if (isEmpty()) {
63         cout << "Queue is empty!" << endl;
64         return -1;
65     }
66     return rear->next->data;
67 }
68
69 // Display the elements of the circular queue
70 void display() {
71     if (isEmpty()) {
72         cout << "Queue is empty!" << endl;
73         return;
74     }
75
76     Node* temp = rear->next;
77     cout << "Circular Queue elements: ";
78     do {
79         cout << temp->data << " ";
80         temp = temp->next;
81     } while (temp != rear->next);
82     cout << endl;
83 }
84
85 ~CircularQueue() {

```

\*/

```

84
85 ~CircularQueue() {
86     if (!isEmpty()) {
87         Node* current = rear->next;
88         rear->next = nullptr; // Break the circle to prevent infinite loop
89         while (current != nullptr) {
90             Node* nextNode = current->next;
91             delete current;
92             current = nextNode;
93         }
94     }
95 }
96 };
97
98 int main() {
99     CircularQueue q;
100
101     q.enqueue(10);
102     q.enqueue(20);
103     q.enqueue(30);
104     q.enqueue(40);
105
106     q.display();
107
108     cout << "Front element is: " << q.peek() << endl;
109
110     q.dequeue();
111     q.dequeue();
112
113     q.display();
114
115     q.enqueue(50);
116     q.enqueue(60);
117
118     q.display();
119
120     return 0;
121 }

```

OUTPUT:

```

10 inserted into the queue.
20 inserted into the queue.
30 inserted into the queue.
40 inserted into the queue.
Circular Queue elements: 10 20 30 40
Front element is: 10
10 removed from the queue.
20 removed from the queue.
Circular Queue elements: 30 40
50 inserted into the queue.
60 inserted into the queue.
Circular Queue elements: 30 40 50 60
PS C:\Users\User\OneDrive\Desktop\DSA>

```

## QUESTION NO 2

SOURCE CODE:

```
1  #include <iostream>
2  using namespace std;
3
4  struct Node {
5      int data;
6      Node* next;
7      Node* prev;
8
9      Node(int val) {
10         data = val;
11         next = nullptr;
12         prev = nullptr;
13     }
14 };
15
16 class Deque {
17 private:
18     Node* front;
19     Node* rear;
20
21 public:
22     Deque() {
23         front = nullptr;
24         rear = nullptr;
25     }
26
27     bool isEmpty() {
28         return front == nullptr;
29     }
30
31     void insertFront(int value) {
32         Node* newNode = new Node(value);
33         if (isEmpty()) {
34             front = rear = newNode;
35         } else {
36             newNode->next = front;
37             front->prev = newNode;
38             front = newNode;
39         }
40         cout << value << " inserted at front." << endl;
41     }
42
43     void insertRear(int value) {
44         Node* newNode = new Node(value);
45         if (isEmpty()) {
```

```

46         front = rear = newNode;
47     } else {
48         rear->next = newNode;
49         newNode->prev = rear;
50         rear = newNode;
51     }
52     cout << value << " inserted at rear." << endl;
53 }
54
55 void deleteFront() {
56     if (isEmpty()) {
57         cout << "Deque Underflow! Queue is empty." << endl;
58         return;
59     }
60     Node* temp = front;
61     cout << temp->data << " deleted from front." << endl;
62     front = front->next;
63     if (front != nullptr) {
64         front->prev = nullptr;
65     } else {
66         rear = nullptr; // List became empty
67     }
68     delete temp;
69 }
70
71 void deleteRear() {
72     if (isEmpty()) {
73         cout << "Deque Underflow! Queue is empty." << endl;
74         return;
75     }
76     Node* temp = rear;
77     cout << temp->data << " deleted from rear." << endl;
78     rear = rear->prev;
79     if (rear != nullptr) {
80         rear->next = nullptr;
81     } else {
82         front = nullptr; // List became empty
83     }
84     delete temp;
85 }
86

```

```

86
87     void display() {
88         if (isEmpty()) {
89             cout << "Deque is empty!" << endl;
90             return;
91         }
92         Node* temp = front;
93         cout << "Deque elements: ";
94         while (temp != nullptr) {
95             cout << temp->data << " ";
96             temp = temp->next;
97         }
98         cout << endl;
99     }
100
101     ~Deque() {
102         while (!isEmpty()) {
103             deleteFront();
104         }
105     }
106 };
107
108 int main(){
109     Deque dequeue;
110     dequeue.insertFront(4);
111     dequeue.insertRear(5);
112     dequeue.insertFront(6);
113     cout<<dequeue.isEmpty()<<endl;
114     dequeue.deleteFront();
115     dequeue.display();
116     return 0;
117 }

```

OUTPUT:

```

cde_using_dll
4 inserted at front.
5 inserted at rear.
6 inserted at front.
0
6 deleted from front.
Deque elements: 4 5
4 deleted from front.
5 deleted from front.
PS C:\Users\User\OneDrive\Desktop\DSA>

```

### QUESTION NO 3

SOURCE COCE:

```
1  #include <iostream>
2  using namespace std;
3
4  class Node {
5  public:
6      int data;
7      Node* next;
8      Node* prev;
9
10     Node(int val) : data(val), next(nullptr), prev(nullptr) {}
11 };
12
13 class DoublyLinkedList {
14 public:
15     Node* head;
16     Node* tail;
17
18     DoublyLinkedList() : head(nullptr), tail(nullptr) {}
19
20     void append(int val) {
21         Node* newNode = new Node(val);
22         if (!head) {
23             head = tail = newNode;
24         } else {
25             tail->next = newNode;
26             newNode->prev = tail;
27             tail = newNode;
28         }
29     }
30
31     void print() {
32         Node* temp = head;
33         while (temp != nullptr) {
34             cout << temp->data << " ";
35             temp = temp->next;
36         }
37         cout << endl;
38     }
39
40     DoublyLinkedList concatenate(const DoublyLinkedList& other) const {
41
42         DoublyLinkedList result;
43
44         Node* temp = this->head;
```

```

46         while(temp!=nullptr){
47             result.append(temp->data);
48             temp=temp->next;
49         }
50
51         temp = other.head;
52
53         while(temp!=nullptr){
54             result.append(temp->data);
55             temp=temp->next;
56         }
57
58         return result;
59     }
60 }
61
62 };
63
64 int main(){
65     DoublyLinkedList L ;
66     for (int i = 2; i <= 10; i += 2) L.append(i);
67
68     DoublyLinkedList M;
69     for (int i = 1; i <= 9; i += 2) M.append(i);
70
71     DoublyLinkedList N = L.concatenate(M);
72
73     cout<<"The concatenated list N is : ";
74     N.print();
75     cout<<endl;
76
77
78     return 0;
79 }

```

OUTPUT:

```

Concatenation }
The concatenated list N is : 2 4 6 8 10 1 3 5 7 9

```

## QUESTION NO 4

SOURCE CODE:

```
61
62     void sortDescending(){
63         if (!head) return ;
64
65         bool swapped;
66         Node* curr;
67
68         do{
69             swapped = false;
70             curr = head;
71             while(curr->next != nullptr){
72                 if (curr->data < curr->next->data){
73                     int temp = curr->data;
74                     curr->data = curr->next->data;
75                     curr->next->data = temp;
76                     swapped = true;
77                 }
78                 curr = curr->next;
79             }
80         }
81         while(swapped);
82
83     }
84
85 };
86
87 int main(){
88     DoublyLinkedList L ;
89     for (int i = 2; i <= 10; i += 2) L.append(i);
90
91     DoublyLinkedList M;
92     for (int i = 1; i <= 9; i += 2) M.append(i);
93
94     DoublyLinkedList N = L.concatenate(M);
95     N.sortDescending();
96
97     cout<<"The concatenated and sorted list N is : ";
98     N.print();
99     cout<<endl;
100 }
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA> cd "c:\Users\User\OneDrive\Desktop\DSA"
The concatenated and sorted list N is : 10 9 8 7 6 5 4 3 2 1
```

## QUESTION NO 5

### SOURCE CODE:

```
BrowserHistory.cpp > main()
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  struct Node {
6      string data;
7      Node* next;
8      Node* prev;
9
10     Node(string url) {
11         data = url;
12         next = nullptr;
13         prev = nullptr;
14     }
15 };
16
17 class BrowserHistory {
18 private:
19     Node* head;
20     Node* tail;
21     Node* current;
22
23 public:
24     BrowserHistory(string homepage) {
25         Node* newNode = new Node(homepage);
26         head = tail = current = newNode;
27     }
28
29     void visit(string url) {
30         Node* newNode = new Node(url);
31
32         if (current->next != nullptr) {
33             Node* temp = current->next;
34             current->next = nullptr;
35
36             while (temp != nullptr) {
37                 Node* nextNode = temp->next;
38                 delete temp;
39                 temp = nextNode;
40             }
41         }
42
43         current->next = newNode;
44         newNode->prev = current;
45     }
```

```

46     current = newNode;
47     tail = current;
48 }
49
50 string back(int steps) {
51     while (steps > 0 && current->prev != nullptr) {
52         current = current->prev;
53         steps--;
54     }
55     return current->data;
56 }
57
58 string forward(int steps) {
59     while (steps > 0 && current->next != nullptr) {
60         current = current->next;
61         steps--;
62     }
63     return current->data;
64 }
65
66 ~BrowserHistory() {
67     Node* temp = head;
68     while (temp != nullptr) {
69         Node* nextNode = temp->next;
70         delete temp;
71         temp = nextNode;
72     }
73 }
74 };
75
76 int main() {
77
78     BrowserHistory browser("leetcode.com");
79
80     browser.visit("google.com");
81     browser.visit("facebook.com");
82     browser.visit("youtube.com");
83
84     cout << "Back 1: " << browser.back(1) << endl;
85
86     cout << "Back 1: " << browser.back(1) << endl;

```

```
87
88     cout << "Forward 1: " << browser.forward(1) << endl;
89
90     browser.visit("linkedin.com");
91
92     cout << "Forward 2: " << browser.forward(2) << endl;
93
94     cout << "Back 2: " << browser.back(2) << endl;
95
96     cout << "Back 7: " << browser.back(7) << endl;
97
98     return 0;
99 }
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA> cd "c:\Users\User\OneDrive\Desktop\DSA"
Back 1: facebook.com
Back 1: google.com
Forward 1: facebook.com
Forward 2: linkedin.com
Back 2: google.com
Back 7: leetcode.com
PS C:\Users\User\OneDrive\Desktop\DSA> 
```